

Class Overview

In Class:

- Model Checking
- Technical Reviews
- Genetic Algorithms & Testing (If time permits)

After Class Office Hours: Homework Help

Homework (Currently Assigned):

- 04-08: Metamorphic Testing
- 04-15: Testing with LLMs
- 04-23: Half-Semester Project

Clicker Points (Current): 4

Note: High volume of early reviews coming in!

1

Clicker Point Options Total Points: 4

Choices (3pts each):

- Final Presentation Instead of Paper Exam
- Extra Credit (5 points)
- Re-do one assignment with no penalty
- Other?

We will “shop” for our choices on the 15th

2

Tentative Remaining Schedule

Week	Plan
04/08 (Today)	Reviews, Model Checking, Search-Based Testing (if time)
04/15	Search-Based Testing (most likely) Exploratory Testing, Final Exam Review
04/22*	Final Assessment, Half-Semester Project Due
04/29	Half-Semester Project Presentations
* No extensions past this point	

3

Course Eval & Extra Credit

- Available on Friday, April 10 through Friday, April 24.
- If the following percent of people complete the survey:
 - **80%**, then **3pt** (out of 250) is added to the $\frac{1}{2}$ semester project.
 - **90%**, then **4pt** (out of 250) is added to the $\frac{1}{2}$ semester project.
 - **100%**, then **5pt** (out of 250) is added to the $\frac{1}{2}$ semester project.

Things I really want to hear about:

- Should the book be required?
- Are the early reviews helpful? Extensions?
- Should the clickers be used more or less?

4

Model Checking & Race Conditions

5

Race Conditions

- Given a class, *IOThread*, it will have two variables:
 - A local ID (e.g., `private int localID`), and
 - A global ID (e.g., `static int ID`)
- When class *IOThread* is created, it will
 1. Start a thread,
 2. Assign the constructor argument to *localID*,
 3. Assign the *localID* to *ID* (i.e., the global ID), and
 4. Return that is has set its IDs, when requested (e.g., by the function call `IDSet()`)

Note: The **last** *IOThread* class to set *ID*
has set it for **every** *IOThread* class.

6

Testing Race Conditions

```
IDThread thread1 = new IDThread(10);
```

Start one thread

```
while(thread1.IDSet() == false)
```

```
    Thread.sleep(500);
```

Wait until it
has set its ID

```
System.out.println(thread1.getLocalID() +
```

```
    " ID1: " + thread1.getGlobalID());
```

Print out the
local ID and
global ID

What is the output?

"10 ID1: 10"

7

Testing Race Conditions

```
IDThread thread1 = new IDThread(10);
```

```
IDThread thread2 = new IDThread(20);
```

Start two threads

```
while(thread1.IDSet() == false ||
```

```
    thread2.IDSet() == false)
```

```
    Thread.sleep(500);
```

Wait until
both have set
their IDs

```
System.out.println(thread1.getLocalID() +
```

```
    " ID1: " + thread1.getGlobalID());
```

```
System.out.println(thread2.getLocalID() +
```

```
    " ID2: " + thread2.getGlobalID());
```

Print out each
one's local ID
and global ID

What is the output?

8

Race Condition Output

- In the cases where **thread1** completes *last*, the output is:

10 ID1: 10

20 ID2: 10

- In the cases where **thread2** completes *last*, the output is:

10 ID1: 20

20 ID2: 20

What is the problem with this?

9

Race Condition Output

Demo

10

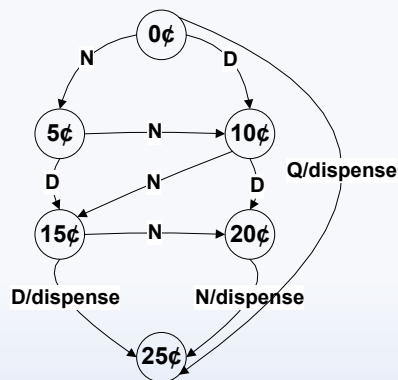
Race Conditions are *Bad!*

- Debugging race conditions is *hard*:
 - Breakpoints can change the result
 - Print statements can change the result
- Race conditions occur *infrequently*:
 - Unlikely to show up in testing
 - Can cause issues with:
 - System crashes,
 - Unexpected behavior,
 - Security issues (e.g., privilege escalation),
 - Networking issues, and even
 - Impact safety-critical systems (e.g, Therac-25)

**Race conditions are hard to catch and hard to fix.
So, what do we do?
Let's model the problem!**

11

State Machines: Antique Vending Machine

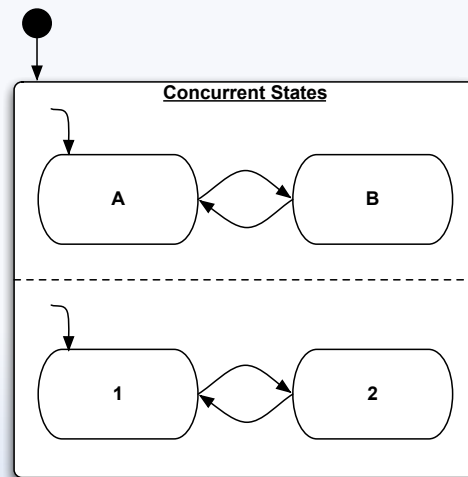


Event sequences:

1. N D D
2. D N D
3. D D N
4. N N N D
5. N N D N
6. N D N N
7. D N N N
8. N N N N N
9. Q

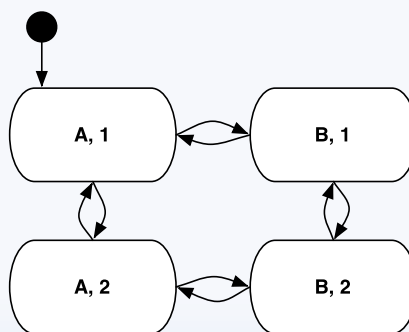
12

Orthogonal Regions



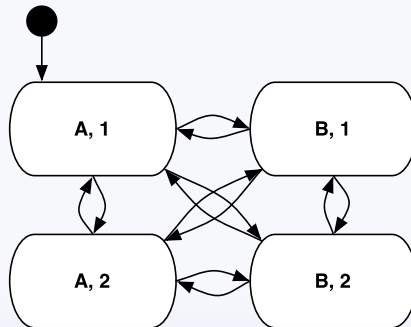
13

Orthogonal Regions



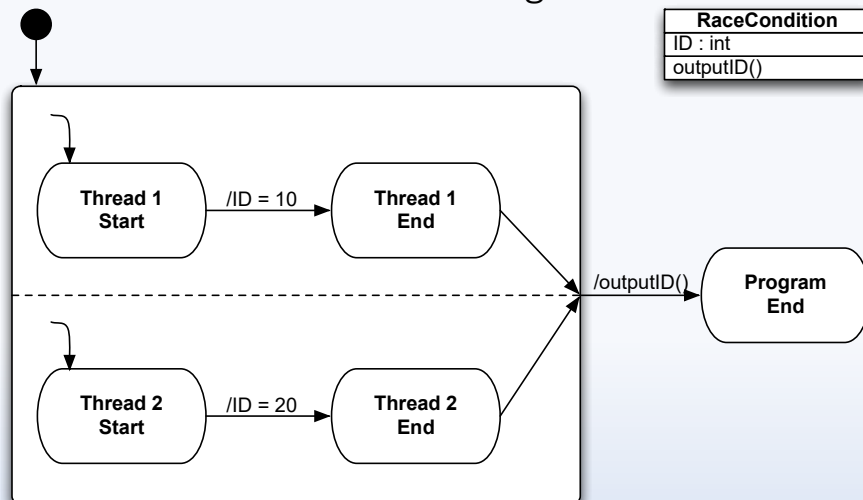
14

Orthogonal Regions



15

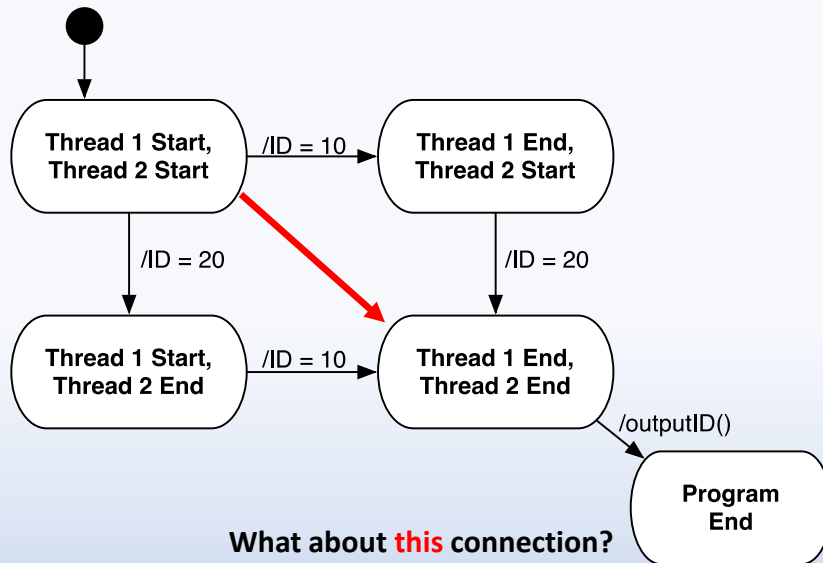
Race Condition Modeling in Statechart



What ID is output?

16

Race Condition Modeling in FSA



17

Model Checking

18

Outline

- Model Checking
- SPIN & Promela
 - SPIN Uses
 - Overview of SPIN & Promela
 - Promela Language
 - Data type
 - Operators & Expressions
 - Selection Statements
 - Repetition Statements
 - Concurrency
 - Assertion
 - Linear Temporal Logic

19

Model Checking

- Verifies models by exhaustively and automatically by:
 - Identifying all states in a model,
 - Visit and/or traverse all states in the model,
 - Ensure specific properties are maintained throughout the time in the model.

20

SPIN & Promela

- **SPIN**: An automated tool for verifying systems*
- Operates on **Promela** (**P**rocess **M**eta **L**anguage) source files
- Typically operates in:
 - **Simulation** Mode, or
 - **Verification** Mode.

* Installation instructions here: <http://spinroot.com/spin/Man/README.html>

21

SPIN Simulation Mode



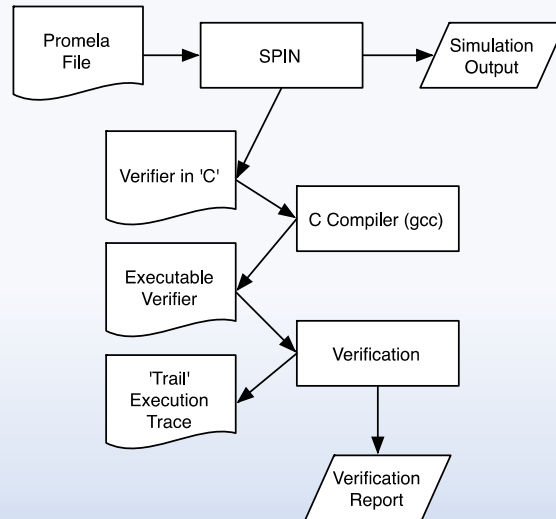
- Randomly selects a path to 'simulate' for output

Command line usage:

```
spin example.pml
```

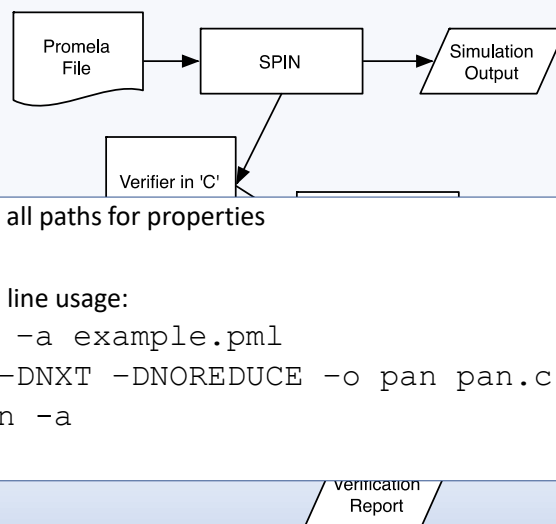
22

SPIN Verification Mode



23

SPIN Verification Mode



24

Promela Language

Similar to C, but with some
big differences.

25

Promela: Data Types*

Types	Values	Bits
bit	0, 1	1
bool	true, false	1
byte	0 ... 255	8
short	-32768 ... 32767	16
int	$-2^{31} \dots 2^{31}-1$	32
unsigned [VAR]:x	0 ... 2^x-1	X (1 ... 32)

Big Difference 1: There are *no* floating point values (e.g., float or double)

*Also included are: **chan**, **pid**, and **mttype**. The chan data type is a pre-defined data type for message passing, the mtype type is used to define symbolic names for numeric constants, and the pid type is the type of `_pid` which is a local read-only variable that stores the process ID for the current process.

26

Promela: Operators & Expressions

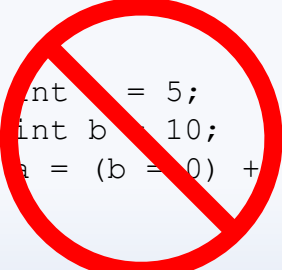
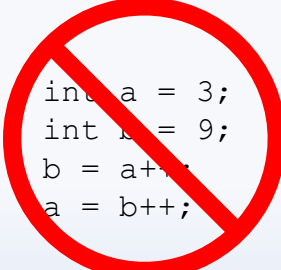
Big Difference 2: No Side Effects!

<code>int a = 5;</code>	<code>int a = 3;</code>
<code>int b = 10;</code>	<code>int b = 9;</code>
<code>a = (b = 0) + 5;</code>	<code>b = a++;</code>
	<code>a = b++;</code>

27

Promela: Operators & Expressions

Big Difference 2: No Side Effects!

 <code>int a = 5;</code>	 <code>int a = 3;</code>
<code>int b = 10;</code>	<code>int b = 9;</code>
<code>a = (b = 0) + 5;</code>	<code>b = a++;</code>
	<code>a = b++;</code>

28

Promela: Selection Statements

C version:

```
int a = 5;
int b = 10;
if(a == 5) {
    printf("a == 5\n");
} else if(b == 10) {
    printf("b == 10\n");
} else if(a == 5 && b == 10) {
    printf("a == 5 && b == 10\n");
}
```

Output:

```
a == 5
```

29

Promela: Selection Statements

C version:

```
int a = 5;
int b = 10;
if(a == 5) { printf("a == 5\n"); }
if(b == 10) {
    printf("b == 10\n");
}
if(a == 5 && b == 10) {
    printf("a == 5 && b == 10\n");
}
```

Output:

```
a == 5
b == 10
a == 5 && b == 10
```

30

Promela: Selection Statements

Promela version:

```
int a = 5;
int b = 10;
if
:: a == 5 -> printf("a == 5\n")
:: b == 10 -> printf("b == 10\n")
:: a == 5 && b == 10 ->
    printf("a == 5 && b == 10\n")
fi
```

Output:

a == 5

or...

b == 10

or...

a == 5 && b == 10

Big Difference 3: Any possible path can be taken!

31

Promela: Repetition Statements

Promela:

```
int N = 10;
int count = 1;
do
:: count <= N -> printf("%d ", count ); count++
:: count > N -> break
od;
```

Output:

1 2 3 4 6 7 8 9 10

Big Difference 4: Only 'do' loops exist*.

*And **goto**, but we won't use that.

32

Promela: Concurrency

Promela:

```
active proctype A() {
    printf("Output from A\n")
}
active proctype B() {
    printf("Output from B\n")
}
```

Output:

```
Output from A
Output from B
```

or...

```
Output from B
Output from A
```

Big Difference 5: Parallelism is easy *and* intended.

33

Promela: Concurrency

Promela:

```
active [2] proctype A() {
    printf("Output from A, process = %d\n", _pid)
}
```

Output:

```
Output from A, process = 0
Output from A, process = 1
```

or...

```
Output from A, process = 1
Output from A, process = 0
```

34

Promela: Concurrency

Promela:

```
proctype A() {
    printf("Output from A, process = %d\n", _pid)
}
init {
    atomic {
        run A();
        run A()
    }
}
```

Output:

```
Output from A, process = 1
Output from A, process = 2
```

or...

```
Output from A, process = 2
Output from A, process = 1
```

35

Verification in SPIN and Promela

There are 2 primary ways of performing verification using

SPIN:

1. Assertions
2. Linear Temporal Logic

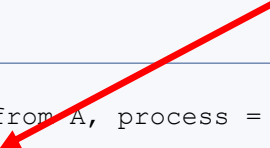
36

Promela: Assertions

Promela:

```
proctype A() {
    printf("Output from A, process = %d\n", _pid)
    assert(_pid < 2)
}
init {
    atomic {
        run A();
        run A()
    }
}
```

Must always be true!



37

Promela: Linear Temporal Logic

Promela (at the top of file):

```
ltl [name] { [LTL expression] }
```

```
ltl myProperty { [] (ID == 10) }
```

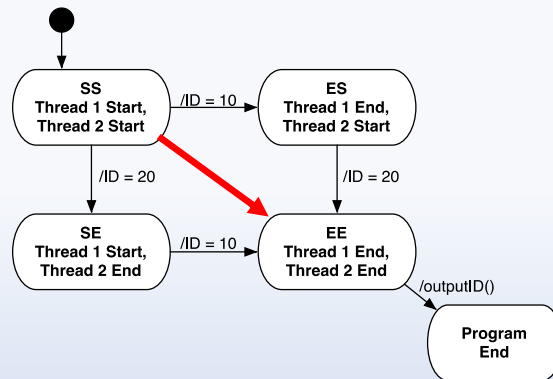
Operators:

$\Box$	=	"always":	$[]$
$\bigcirc$	=	"next":	X
$\Diamond$	=	"eventually":	$\langle \rangle$
U	=	"until":	U
$\wedge$	=	"and":	$\&\&$
$\vee$	=	"or":	$ $
$\rightarrow$	=	"implies":	$\rightarrow$
$\neg$	=	"not":	$!$

38

So, what now?

Let's model our FSA race condition in Promela!



39

FSM to Promela

FSMs are made up of:

1. States,
2. Transitions, with
 - a) Input Events, and
 - b) Output Actions,
3. An initial State, and
4. A Set of final States

40

FSM to Promela

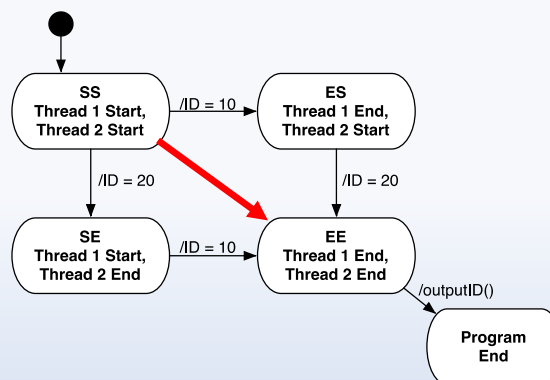
Steps:

1. Create all states (i.e., Q) as *bool* variables in Promela
2. Create an *if fi* statement that:
 - A. For each state, and for each transition from the current state:
 - i. Sets the new state
 - ii. Resets the old state
 - iii. Sets the necessary transition, if it exists
 - iv. Perform transition action, if it exists
3. Set the initial state in *init* to true and run
4. Create transition and class data
5. Create a proctype for the FSM

41

FSM to Promela

Step 1: Create all states (i.e., Q) as *bool* variables in Promela



42

FSM to Promela

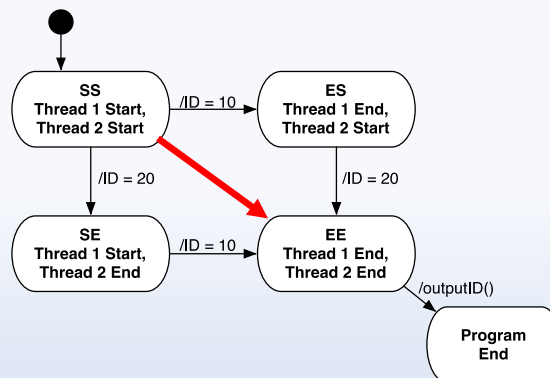
Step 1: Create all states (i.e., Q) as *bool* variables in Promela

```
bool SS = false;
bool ES = false;
bool SE = false;
bool EE = false;
bool END = false;
```

43

FSM to Promela

Step 2: Create an *if fi* statement



44

FSM to Promela

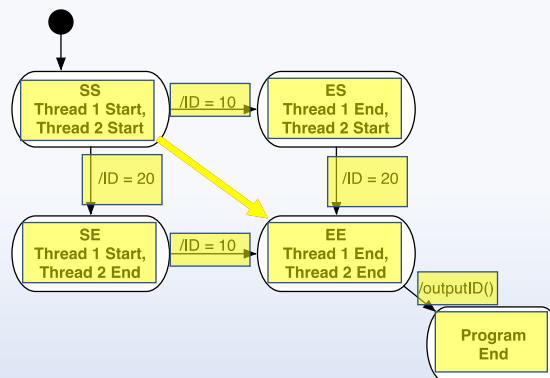
Step 2: Create an *if fi* statement

```
if
/* ??? */
fi
```

45

FSM to Promela

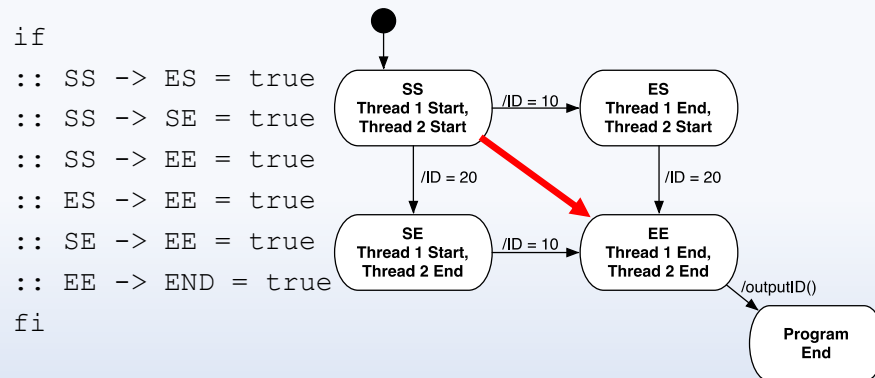
Step 2.A: For each state, and for each transition from the current state:



46

FSM to Promela

Step 2.A.i: Sets the new state:



47

FSM to Promela

Step 2.A.ii: Resets the old state:

```

if
:: SS -> ES = true; SS = false
:: SS -> SE = true; SS = false
:: SS -> EE = true; SS = false
:: ES -> EE = true; ES = false
:: SE -> EE = true; SE = false
:: EE -> END = true; EE = false
fi

```

48

FSM to Promela

Step 2.A.iii: Sets the necessary event, if it exists:

```
if
:: SS -> ES = true; SS = false; event = -1
:: SS -> SE = true; SS = false; event = -1
:: SS -> EE = true; SS = false; event = -1
:: ES -> EE = true; ES = false; event = -1
:: SE -> EE = true; SE = false; event = -1
:: EE -> END = true; EE = false; event = -1
fi
```

49

FSM to Promela

Step 2.A.iv: Perform transition action, if it exists:

```
if
:: SS -> ES = true; SS = false; event = -1; ID = 10; printf("< >")
:: SS -> SE = true; SS = false; event = -1; ID = 20; printf("< >")
:: SS -> EE = true; SS = false; event = -1; ID = -1; printf("< >")
:: ES -> EE = true; ES = false; event = -1; ID = 20; printf("< >")
:: SE -> EE = true; SE = false; event = -1; ID = 10; printf("< >")
:: EE -> END = true; EE = false; event = -1; printf("<%d>", ID)
fi
```

50

FSM to Promela

Step 3: Set the initial state in *init* to true and run:

```
init{  
    SS = true;  
    run FSA();  
}
```

51

FSM to Promela

Step 4: Create transition and class data:

```
int event = -1;  
int ID = 0;
```

52

FSM to Promela

Step 5: Create a *proctype* for the FSM:

```
proctype FSA() {
    ...
}
```

53

Demo: Race Condition Example

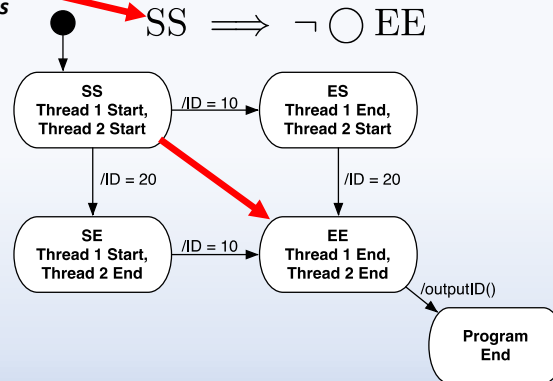
- Ensure **this** connection never occurs:

Includes an implicit *always* that SPIN requires

Is there a simpler version?

$SS \Rightarrow (\Diamond ES \vee \Diamond SE)$

$SS \Rightarrow \neg \bigcirc EE$



54

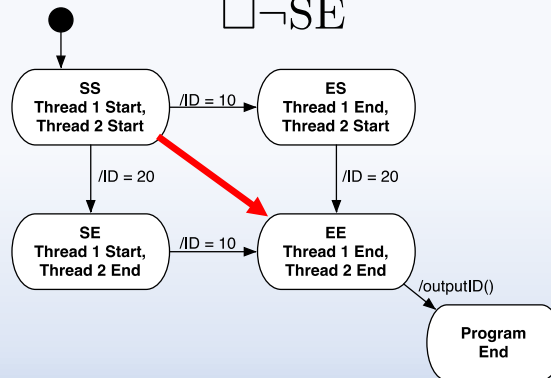
Race Condition Example

- Ensure we know which ID is output?

$$SS \cup ES \cup EE$$

Is there a simpler version?

$$\Box \neg SE$$



55

Race Conditions in SPIN: Original

```

devribyHV2Q:33 - Sources devriby$ ./spin648_mac64 -a simpleracecondition.pml
l1l mytest: [ ((! (SS))) || ((<=> (ES))) || ((<=> (SE)))]
devribyHV2Q:33 - Sources devriby$ gcc -DNOAT -DNOREDUCE -a pan pan.c
devribyHV2Q:33 - Sources devriby$ ./pan -a
pan:1: acceptance cycle (at depth 60)
pan: wrote simpleracecondition.pml.trail

(Spin Version 6.4.8 -- 2 March 2018)
Warning: Search not completed

Full statespace search for:
  never claim      + (mytest)
  assertion violations + (if within scope of claim)
  acceptance cycles + (fairness disabled)
  invalid end states - (disallowed by never claim)

State-vector 36 byte, depth reached 61, errors: 1
  35 states, stored (39 visited)
  0 states, matched
  39 transitions (= visited-matched)
  0 atomic steps
hash conflicts:      0 (resolved)

Stats on memory usage (in Megabytes):
  0.002   equivalent memory usage for states (stored*(State-vector + overhead))
  0.283   actual memory usage for states
128.000   memory used for hash table (-w24)
  0.534   memory used for DFS stack (-m10000)
128.730   total actual memory usage

pan: elapsed time 0.01 seconds
  
```

56

Race Conditions in SPIN: Updated

Left Screenshot (Summary):

```

tion violations + (if within scope of claim)
distance cycles + (fairness disabled)
lid end states - (disabled by never claim)

36 byte, depth reached 40, errors: 0
ates, stored (60 visited)
ates, matched
ansitions (= visited+matched)
omic steps
ts: 0 (resolved)

```

Right Screenshot (Detailed Log):

```

[Spin Version 6.4.8 - 2 March 2008]

full state-space search fun
never claim
assertion violations + (if within scope of state)
acceptance cycles + (if success disabled)
load (id and states) + (if added to never claim)

State search 36 byte, depth reached 40, errors: 0
# states, stored (60 visited)
# states, matched
# transitions (= visited+matched)
# atomic steps
heap conflicts: 0 (resolved)

Stats for memory usage (in Megabytes):
0.000 maximum memory usage for global (extern) state sector - overhead0
0.000 actual memory usage for states
0.000 memory used for book-keeping (-wss)
0.000 memory used for OS stack (-s00000)
0.000 total actual memory usage

unreached in prototype file
FixedAssertionCondition.pd.27, state 8, "assert('B'==0)"
FixedAssertionCondition.pd.35, state 33, "assert('B'==1)"
FixedAssertionCondition.pd.32, state 33, "assert('C'==00000)"
FixedAssertionCondition.pd.36, state 25, "assert = 'C'?"
FixedAssertionCondition.pd.36, state 26, "B == 0?"
FixedAssertionCondition.pd.36, state 27, "assert('C'==1)"
FixedAssertionCondition.pd.38, state 25, "B == 1?"
FixedAssertionCondition.pd.36, state 36, "B == 0?"
FixedAssertionCondition.pd.38, state 31, "assert = 'B'?"
FixedAssertionCondition.pd.38, state 31, "assert('B'==0)"
FixedAssertionCondition.pd.41, state 36, "assert('B'==0)"
FixedAssertionCondition.pd.41, state 42, "assert('C'==0)"
FixedAssertionCondition.pd.44, state 44, "assert('C'==0)"
FixedAssertionCondition.pd.46, state 44, "assert('C'==0)"
FixedAssertionCondition.pd.52, state 56, "assert('C'==0)"
FixedAssertionCondition.pd.52, state 51, "assert('C'==0)"
FixedAssertionCondition.pd.58, state 58, "end"
(27 of 48 states)
unreached in init
(8 of 8 states)
unreached in claim never0
-assert('B'==0, state 38, "end"
(3 of 38 states)

spin elapsed time 0 seconds

```

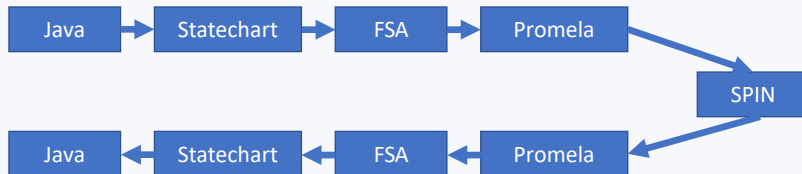
57

What does this get us?

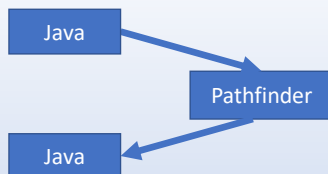
- **Reminder:**
 - Linear Temporal Logic allows us to *specify* properties and behavior
 - LTL doesn't guarantee that the system *follows* those properties
- We can use SPIN & Promela to:
 - Design the system to adhere to the properties specified, and
 - Verify that the system adheres to the properties specified.

58

Path we have taken...



Another path we can take...



59

Java Pathfinder

- Developed at the Robust Software Engineering Group at NASA Ames Research Center
- First development cycle used Spin/Promela translator - 1999
- Added:
 - Backtrackable, state matching JVM – 2000
 - Extension infrastructure (listeners, MJL) – 2004
 - Symbolic execution, choice generators - 2005
- Originally open sourced in 2005 under <http://javapathfinder.sourceforge.net>
- First NASA-developed system hosted on a public site

Now on Github like everything else: <https://github.com/javapathfinder>

60

Simple Example

```
public class Rand {
    public static void main (String[] args) {
        Random random = new Random(42);

        int a = random.nextInt(2);
        System.out.printf("a=%d\n", a);

        int b = random.nextInt(3);
        System.out.printf(" b=%d\n", b);

        int c = a/(b+a -2);
        System.out.printf(" c=%d\n", c);
    }
}
```

61

Simple Example

```
public class Rand {
    public static void main (String[] args) {
        Random random = new Random(42);

        int a = random.nextInt(2);
        System.out.printf("a=%d\n", a);

        int b = random.nextInt(3);
        System.out.printf(" b=%d\n", b);

        int c = a/(b+a -2);
        System.out.printf(" c=%d\n", c);
    }
}
```

Output

```
> java Rand
a=1
 b=0
 c=-1
>
```

62

Example

> jpf Rand

JavaPathfinder core system v8.0 (rev) - (C) 2005-2014 United States Government. All rights reserved.

===== system under test

Rand.main()

===== search started:

11/19/18 9:40 AM

a=1

b=0

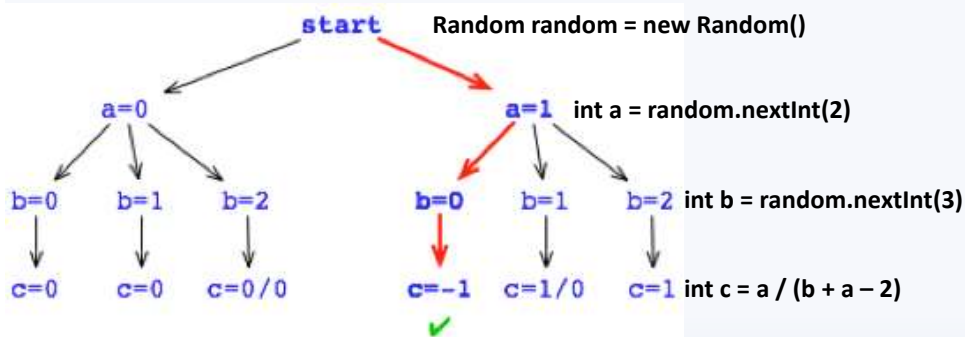
c=-1

===== results

no errors detected

63

Example



One execution corresponds to one path

64

Example

```
> jpf +cg.enumerate_random=true Rand
JavaPathfinder core system v8.0 (rev ) - (C) 2005-2014 United States Government. All rights reserved.
```

```
===== system under test
Rand.main()

===== search started: 11/19/18 9:45 AM
a=0
b=0
  c=0
b=1
  c=0
b=2

===== error 1
gov.nasa.jpf.vm.NoUncaughtExceptionsProperty
java.lang.ArithmeticException: division by zero
    at Rand.main(Rand.java:13)
```

65

Example

```
> jpf +cg.enumerate_random=true Rand
JavaPathfinder core system v8.0 (rev ) - (C) 2005-2014 United States Government. All rights reserved.
```

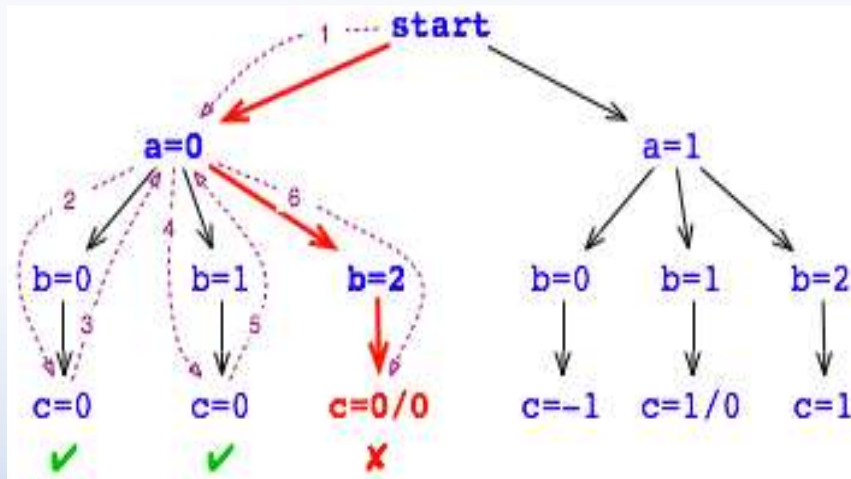
```
===== system under test
Rand.main()

===== error 1
gov.nasa.jpf.vm.NoUncaughtExceptionsProperty
java.lang.ArithmeticException: division by zero
    at Rand.main(Rand.java:13)

===== error 1
gov.nasa.jpf.vm.NoUncaughtExceptionsProperty
java.lang.ArithmeticException: division by zero
    at Rand.main(Rand.java:13)
```

66

Example



67

Testing Race Conditions

```
IDThread thread1 = new IDThread(10);
IDThread thread2 = new IDThread(20);
```

Start two threads

```
while(thread1.IDSet() == false ||
      thread2.IDSet() == false)
    Thread.sleep(500);
```

Wait until
both have set
their IDs

```
System.out.println(thread1.getLocalID() +
    " ID1: " + thread1.getGlobalID());
System.out.println(thread2.getLocalID() +
    " ID2: " + thread2.getGlobalID());
```

Print out each
one's local ID
and global ID

What is the output?

68

Race Condition Output

- In the cases where `thread1` completes *last*, the output is:

10 ID1: 10
20 ID2: 10

- In the cases where `thread2` completes *last*, the output is:

10 ID1: 20
20 ID2: 20

What is the problem with this?

69

Race Conditions in Java Pathfinder: Original

The screenshot displays two windows from the Java Pathfinder application. The left window shows the output of two threads, `thread1` and `thread2`, which are interleaved, indicating a race condition. The right window shows the program's internal state and search results.

```

deviamp@N20:~$ ./deviamp5 /Users/deviamp/Desktop/jpf-core-master/bin/jpf IDThreadOriginal
javapathfinder core system v8.0 (rev 3) - (C) 2005-2014 United States Government. All rights reserved.

===== system under test
IDThreadOriginal.main()

===== search started: 12/1/21 7:41 PM

10 ID1: 20
20 ID2: 20
10 ID1: 20
20 ID2: 20
10 ID1: 20
20 ID2: 20
10 ID1: 10
20 ID2: 20
10 ID1: 10
20 ID2: 10
10 ID1: 10
20 ID2: 10
10 ID1: 20
20 ID2: 20
10 ID1: 10
20 ID2: 10
10 ID1: 10
20 ID2: 10
10 ID1: 20
20 ID2: 20
10 ID1: 10
20 ID2: 10

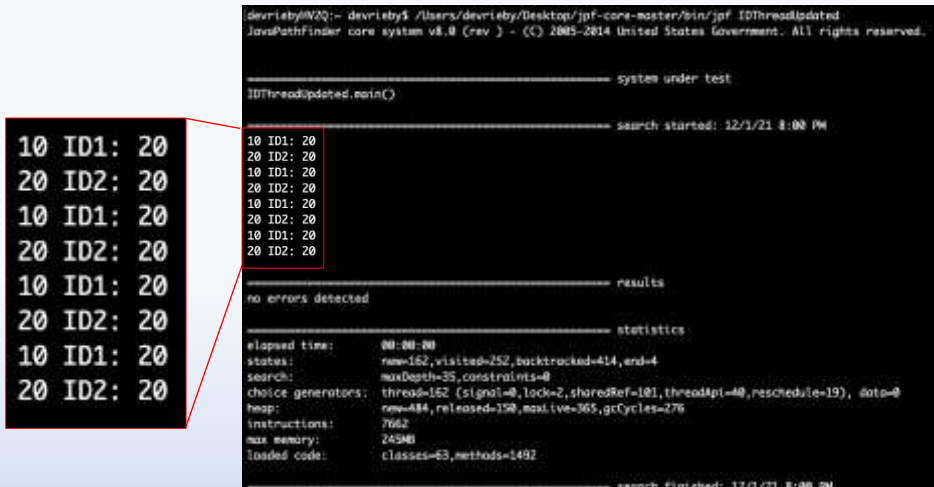
===== results
no errors detected

===== statistics
elapsed time: 00:00:00
states: new=183,visited=274,backtracked=437,end=8
search: maxDepth=34,constraints=0
choice generators: thread=182 (signal=0,lock=2,sharedRef=107,threadId=44,reschedule=25), data=0
heap: new=556,released=277,maxLive=365,gCycles=286
instructions: 8192
new memory: 24996
loaded code: classes=63,methods=5482

===== search finished: 12/1/21 7:41 PM
  
```

70

Race Conditions in Java Pathfinder: Updated



```
devrieby@W2Q:~ devrieby$ ./Users/devrieby/Desktop/jpf-core-master/bin/jpf.IDThreadUpdated
JavaPathfinder core system v8.0 (rev ) - (C) 2005-2014 United States Government. All rights reserved.

===== system under test
IDThreadUpdated.main()

===== search started: 12/1/21 8:00 PM

10 ID1: 20
20 ID2: 20
10 ID1: 20
20 ID2: 20
10 ID1: 20
20 ID2: 20
10 ID1: 20
20 ID2: 20
10 ID1: 20
20 ID2: 20
10 ID1: 20
20 ID2: 20

===== results
no errors detected

===== statistics
elapsed time: 00:00:00
status: new=162,visited=252,backtracked=414,end=4
search: maxDepth=35,constraints=0
choice generators: thread=162 (signal=0,lock=2,sharedRef=101,threadPrt=0,reschedule=19), data=0
heap: new=484,released=150,maxLive=365,gcCycles=276
instructions: 7662
max memory: 245M
loaded code: classes=63,methods=1492

===== search finished: 12/1/21 8:00 PM
```

71

Conclusion

72